

COMPLETE WEBSITE DOCUMENTATION

ShreeGanesh Plants-store

User Manual, Functional Specification & Technical Reference

Document scope: Flask + MySQL e-commerce website ka verified functional aur technical documentation. Current source code snapshot ke anusar taiyar kiya gaya.

Version 1.0 | 03 October 2026

Prepared for project handover, maintenance, demonstration and future development

Document Control

Field	Detail
Project	ShreeGanesh Plants-store
Application type	Responsive plant and gardening e-commerce web application
Backend	Python Flask 2.3.0
Database	MySQL through PyMySQL 1.1.2
Frontend	HTML5, Jinja2, CSS3, JavaScript, Font Awesome
Primary entry point	app.py
Catalog baseline	73 curated products across 9 categories
Document status	Current implementation reference
Repository	github.com/bhartie8484-prog/ShreeGanesh-Plants-store

How to Use This Document

- End users: Sections 3 to 7 explain browsing, account, cart, checkout, orders and support pages.
- Developers: Sections 8 to 15 cover architecture, routes, database, catalog, configuration and maintenance.
- Evaluators: Sections 1, 2 and 16 summarize scope, completed features, known constraints and future roadmap.

Important: The checkout and payment screens are demonstration flows. They do not connect to a real payment gateway and do not charge money.

Contents

1. Executive Summary
2. Scope and Feature Inventory
3. Website Navigation and Public Pages
4. Account and Authentication
5. Product Discovery and Catalog
6. Cart, Checkout and Payment
7. Orders, Tracking and Customer Support
8. System Architecture
9. Application Routes
10. Database Design
11. Catalog and Image Management
12. Frontend Design and JavaScript

13. Installation and Local Setup
14. Configuration and Security
15. Testing and Troubleshooting
16. Known Limitations and Roadmap
17. Appendix A. Folder Structure
18. Appendix B. Operational Checklist

1. Executive Summary

ShreeGanesh Plants-store ek full-stack educational e-commerce website hai jo plants, gardening tools aur related products ko browse aur order karne ka responsive experience deti hai. Application server-side rendering ke liye Flask/Jinja2, persistent data ke liye MySQL, aur interaction/design ke liye CSS aur vanilla JavaScript use karti hai.

Current system mein public catalog browsing, nine-category filtering, product detail pages, user registration/login, persistent shopping cart, address collection, demo payment outcomes, stock validation, order creation, order history, public order tracking, contact storage, policies, FAQ aur custom error pages implemented hain.

Current baseline: Catalog code 73 products supply karta hai: Outdoor 9 items; baaki Indoor, Flowering, Herbal, Fruit, Vegetable, Decorative, Hanging aur Gardening Tools mein 8-8 items. Price range Rs.149 se Rs.1,899 hai.

Primary Objectives

- Customers ko category-wise plants aur tools discover karne dena.
- Secure password hashing aur session-based login ke saath customer account dena.
- Cart se checkout, simulated payment aur confirmed order tak complete shopping journey dikhana.
- Local product images, responsive layouts aur mobile navigation ke saath polished storefront banana.
- MySQL-based order, payment, stock aur contact data persistence demonstrate karna.

2. Scope and Feature Inventory

Module	Implemented capability	Status
Storefront	Hero, featured products, category navigation, promotions, trust messaging	Complete
Catalog	All products, nine filters, product detail, local/remote image support	Complete
Authentication	Registration, login, logout, hashed passwords, sessions	Complete
Cart	Add, increment duplicate item, update quantity, remove, totals	Complete
Checkout	Delivery details, PIN validation, shipping fee rules	Complete
Payment	UPI/card/COD selection and simulated success/failure	Demo only
Orders	Order number, items, payment record, stock deduction, history	Complete
Support	Contact form, FAQ, about, policies, order tracking	Complete
Administration	Admin dashboard and UI-based product/order management	Not included

3. Website Navigation and Public Pages

3.1 Global Layout

base.html common navigation, flash messages, footer links, responsive menu aur shared CSS/JavaScript load karta hai. Logged-in state ke basis par cart, logout aur orders links dikhte hain; guest state mein login aur register links milte hain.

- Main navigation: Home, All Plants, About, Contact.
- Authenticated navigation: Cart, Orders and Logout.
- Footer shop links: Flowering, Herbal, Fruit and Gardening Tools.
- Footer help links: Shipping Policy, Return Policy, FAQ and Track Order.
- Legal links: Privacy and Terms.

3.2 Home Page (/)

Home page available in-stock products ko random order mein load karti hai. System pehle har category se ek representative item choose karta hai, phir remaining products add karke maximum 12 featured products show karta hai. Is approach se home page par category diversity maintain hoti hai.

- Premium hero section aur primary shop call-to-action.
- Visual category navigation for all nine categories.
- Featured/best-selling style product cards with image, category, price and stock cues.
- Direct product details aur add-to-cart actions.
- Promotional and plant-care content sections.

3.3 Informational Pages

Page	URL	Purpose
About	/about	Brand story, healthy plants, expert support and responsible choices.
FAQ	/faq	Frequently asked questions for ordering and plant purchase support.
Contact	/contact	Name, email and message collect karke contacts table mein store karta hai.
Policies	/policy/<slug>	Shipping, returns, privacy and terms content.
Error	404 / 500	Friendly error page with home navigation.

4. Account and Authentication

4.1 Registration

1. User /register page par full name, unique email aur password submit karta hai.
2. Backend Werkzeug generate_password_hash() se password ka hash banata hai.
3. users table mein name, email aur hashed password insert hota hai.
4. Duplicate email par transaction rollback hota hai aur readable error flash hota hai.
5. Successful registration user ko login page par redirect karti hai.

4.2 Login and Session

Login email se user record fetch karta hai aur check_password_hash() se password verify karta hai. Success par session mein user_id aur user_name save hote hain. Logout poori session clear karta hai.

Access control: Cart, checkout, payment, order success aur order history routes logged-in user require karte hain. Unauthorized access ko login/checkout ki taraf redirect kiya jata hai.

4.3 Authentication Data Flow

Input	Processing	Persistent/session result
Registration form	Validation + password hashing	New users row
Login form	Email lookup + hash comparison	user_id and user_name session keys
Logout request	session.clear()	Authenticated state removed

Security Notes

- Passwords plaintext mein store nahi hote; Werkzeug hashing use hoti hai.
- SQL statements parameterized placeholders use karte hain, jo SQL injection risk reduce karte hain.
- Current forms mein CSRF tokens implemented nahi hain; production se pehle Flask-WTF/CSRF protection add karna chahiye.
- Session secret local configuration/environment se strong random value hona chahiye; development fallback production ke liye unsuitable hai.

5. Product Discovery and Catalog

5.1 Supported Categories

Category	Catalog count	Typical products
Indoor	8	Aloe Vera, Snake Plant, Money Plant, ZZ Plant
Outdoor	9	Caladium, Coral Bells, Curry Leaf, Fern, Neem
Flowering	8	Rose, Hibiscus, Jasmine, Marigold
Herbal	8	Tulsi, Mint, Ashwagandha, Lemongrass
Fruit	8	Lemon, Mango, Guava, Papaya
Vegetable	8	Tomato, Chilli, Brinjal, Cucumber
Decorative	8	Aglaonema, Croton, Ficus, Lucky Bamboo
Hanging	8	Golden Money Plant, Spider Plant, Philodendron
Gardening Tools	8	Pruner, spade, rake, gloves, watering can

5.2 Product Listing

/products bina query parameter ke sab in-stock items load karta hai. category query parameter dene par exact category match ke saath filter hota hai. Template active filter ko visually highlight karta hai aur empty-result state bhi provide karta hai.

```
/products
/products?category=Indoor
/products?category=Gardening%20Tools
```

5.3 Product Detail

/product/<id> selected product ka complete record dikhata hai. Missing ID par custom 404 return hota hai. Page product image, name, category, description, current price, stock indication, care/value details aur add-to-cart action provide karti hai.

5.4 Catalog Synchronization

Har request se pehle ensure_catalog() curated catalog ko database ke saath synchronize karta hai. app_meta table catalog_version track karti hai. Version mismatch par existing products update hote hain aur missing products insert hote hain; referenced records ko wholesale delete nahi kiya jata.

Catalog version: Current code catalog_version = 6 use karta hai. Purane version 4 aur 5 se specific Indoor/Outdoor product rename migrations bhi included hain.

6. Cart, Checkout and Payment

6.1 Shopping Cart

Cart database-backed hai, isliye authenticated user ke items requests ke beech persistent rehte hain. users aur products ke combination par unique constraint duplicate row ko रोकता hai; same product dobara add karne par quantity increment hoti hai.

Action	Route	Behavior
View cart	GET /cart	Joined product rows aur calculated total show karta hai.
Add product	GET/POST /add_to_cart/<id>	New row insert ya existing quantity +1.
Update quantity	POST /cart/update/<id>	Minimum 1 validate karke owned row update.
Remove item	POST /cart/remove/<id>	Owned cart row delete.

6.2 Checkout Rules

- Empty cart checkout nahi kar sakta.
- Required fields: full name, phone, address, city, state, six-digit PIN and payment method.
- Subtotal Rs.999 se greater hone par shipping free; otherwise Rs.50.
- Checkout form values temporary session['checkout'] structure mein store hote hain.
- Available methods: UPI, card and cash on delivery (COD).

6.3 Demo Payment and Order Transaction

1. Payment page current cart se total dobara calculate karti hai.
2. User simulated success ya failure choose karta hai; failure par koi order/charge nahi banta.
3. Success se pehle requested quantity ko current product stock ke against validate kiya jata hai.
4. Unique order number PS + date + random suffix se generate hota hai.
5. orders row confirmed status ke saath create hota hai; COD payment pending, other methods paid hote hain.
6. Har cart item order_items mein snapshot price aur quantity ke saath insert hota hai.
7. Product stock deduct hota hai aur payments row transaction ID ke saath create hota hai.
8. User cart clear hota hai, database commit hota hai aur success page open hoti hai.

Payment disclaimer: Payment screen sirf end-to-end workflow testing ke liye fake gateway hai. Card/UPI credentials collect ya store nahi kiye jate.

7. Orders, Tracking and Customer Support

7.1 Order Confirmation and History

Successful payment ke baad `/order-success/<order_number>` sirf authenticated owning user ko order dikhata hai. `/orders` logged-in user ke orders newest-first list karta hai, including total, payment state aur order state.

7.2 Public Order Tracking

`/track-order` par order number uppercase/trim karke lookup hota hai. Match hone par order status, payment status, created date context aur destination city show hoti hai. Invalid number par flash error milta hai.

Privacy observation: Current tracking lookup order number janne wale visitor ko limited order status aur city dikhata hai. Production mein registered email/phone verification ya signed tracking token add karna recommended hai.

7.3 Contact and Policy Support

- Contact submissions contacts table mein timestamp ke saath persist hote hain.
- Shipping policy dispatch 2-4 business days aur delivery 3-7 business days ka guidance deti hai.
- Returns content damaged/incorrect items ko 24 hours mein photos aur unboxing video ke saath report karne ko kehta hai.
- Privacy page order/support purposes ke liye minimal data collection explain karti hai.
- Terms page natural product variation aur demo-payment nature clarify karti hai.

Customer Journey Summary

Stage	Customer action	System outcome
Discover	Home/category browse	In-stock product cards
Evaluate	Open detail page	Description, price, stock and image
Account	Register/login	Authenticated session
Purchase	Cart + checkout	Validated delivery/payment selection
Confirm	Simulate payment	Order, payment and stock transaction
After-sales	Orders/track/contact	Status visibility and support capture

8. System Architecture

Application traditional server-rendered three-layer structure follow karti hai. Browser request Flask route tak jati hai; route MySQL data read/write karke Jinja2 template render karta hai; CSS/JavaScript presentation aur client-side interaction enhance karte hain.

Layer	Components	Responsibility
Presentation	templates/*.html, static/css/style.css, static/js/main.js	Responsive UI, forms, navigation, cards, feedback and search filtering.
Application	app.py	Routing, sessions, validation, business rules, checkout and error handling.
Catalog	catalog.py	Categories and curated product dataset for synchronization.
Persistence	MySQL plant_store	Users, products, carts, orders, payments, contacts and metadata.
Configuration	config/local.py (ignored)	Machine-specific database and secret settings.
Media	static/uploads	Locally served product and marketing images.

Request Lifecycle

1. Browser Flask route request bhejta hai.
2. before_request hook catalog aur evolving order/payment schema ensure karta hai.
3. get_db() request context mein lazy MySQL connection create/reuse karta hai.
4. Route parameterized SQL aur business validation execute karta hai.
5. Jinja2 HTML generate karta hai; static assets browser load karta hai.
6. teardown_appcontext request ke end par connection close karta hai.

Runtime Characteristics

- MySQL connection utf8mb4 charset aur autocommit=False ke saath create hota hai.
- Write flows explicit commit/rollback use karte hain.
- Catalog/schema readiness process globals se repeated work reduce karti hai.
- Application production entry point par debug=False se run hoti hai.

9. Application Routes

Method	Route	Purpose	Access
GET	/	Home/featured products	Public
GET	/products	All/category product listing	Public
GET	/product/<id>	Product detail	Public
GET/POST	/register	Create user account	Public
GET/POST	/login	Authenticate user	Public
GET	/logout	Clear session	Logged-in
GET	/cart	View cart	Logged-in
POST	/cart/update/<id>	Change quantity	Logged-in
POST	/cart/remove/<id>	Remove cart item	Logged-in
GET/POST	/add_to_cart/<id>	Add/increment product	Logged-in
GET/POST	/checkout	Delivery and payment selection	Logged-in
GET/POST	/payment	Demo payment completion	Logged-in
GET	/order-success/<number>	Owned order confirmation	Logged-in
GET	/orders	Owned order history	Logged-in
GET/POST	/track-order	Lookup order number	Public
GET/POST	/contact	Contact message	Public
GET	/about	Brand information	Public
GET	/faq	Frequently asked questions	Public
GET	/policy/<slug>	Policy content	Public

10. Database Design

Table	Purpose	Important relationships/constraints
users	Customer identity and password hash	Unique email; parent of cart/orders
products	Catalog content, price, stock, image, category	Referenced by cart/order_items
cart	Per-user active basket	Unique user_id + product_id; cascade delete
orders	Order header, totals, delivery and status	Belongs to user; unique order_number
order_items	Purchased product line snapshot	Belongs to order and product
payments	Demo transaction ledger	Belongs to order; unique transaction_id
contacts	Customer support messages	Independent timestamped records
app_meta	Catalog migration/version state	String key/value primary key

10.1 Relationship Model

- One user can have many cart rows and many orders.
- One product can appear in many carts and order items.
- One order contains many order items and one or more payment records by schema capability (current flow creates one).
- Deleting a user cascades their cart and orders; deleting an order cascades order items and payments.
- order_items product foreign key intentionally preserves product reference rules without ON DELETE CASCADE.

10.2 Runtime Schema Migration

ensure_store_schema() existing orders table mein order number, payment, shipping fields add karne ka प्रयास करता है. MySQL duplicate-column error 1060 ignore होता है. payments table absent हो तो create होती है. Ye lightweight compatibility mechanism है, full migration framework नहीं.

Recommended: Production growth के लिए Flask-Migrate/Alembic जैसा versioned migration system use करें. Runtime ALTER TABLE को startup deployment step से replace करें.

10.3 Key Business Data

Field	Rule
products.stock	Successful order के वाक़्त ordered quantity से decrease.
orders.status	Current successful flow में confirmed.
orders.payment_status	COD = pending; UPI/card demo success = paid.
orders.order_number	PS + YYMMDD + six uppercase random hex characters.

Field	Rule
payments.transaction_id	TXN + twelve uppercase random hex characters.

11. Catalog and Image Management

11.1 Product Record Shape

catalog.py ke db_products() records database insert order follow karte hain: name, description, price, stock, image filename aur category. Database query results templates mein positional indexes se consumed hote hain.

```
(name, description, price, stock, image, category)
```

11.2 Adding a Product

1. Optimized product image static/uploads folder mein copy karein.
2. catalog.py ke appropriate category group mein unique product tuple add karein.
3. Catalog version number increment karein taaki existing database synchronize ho.
4. Application restart karke products page aur product detail image verify karein.
5. Cart, checkout aur stock deduction path test karein.

11.3 Image Resolution

Templates image string http se start hone par remote URL directly use karte hain; otherwise static/uploads/<filename> build karte hain. Current project majorly local curated image assets use karta hai.

- Recommended web formats: WebP/JPEG for photos, PNG only where transparency needed.
- Consistent aspect ratio and compressed files page stability/performance improve karte hain.
- Filename database/catalog value se exact match hona chahiye.
- Missing images ke liye placeholder strategy maintain karein.

11.4 Upload Support

app.py allowed file extensions png, jpg, jpeg aur gif define karta hai aur UPLOAD_FOLDER static/uploads set karta hai. Current main application mein admin/user upload route implemented nahi hai; configuration future upload feature ke liye prepared hai.

12. Frontend Design and JavaScript

12.1 Design System

style.css green botanical visual language, responsive containers, card grids, rounded controls, hover states, commerce layouts, status pills, flash messages aur mobile navigation implement karta hai. Templates Font Awesome icons use karte hain.

- Desktop and mobile navigation with hamburger control.
- Responsive product/category grids and commerce cards.
- Accessible labels/required attributes on core forms.
- Visual feedback through flash messages and notification behavior.
- Lazy-loaded imagery in major catalog/category sections where specified.

12.2 main.js Responsibilities

Area	Behavior
Navigation	Mobile menu toggle, Escape close, resize cleanup and link-close behavior.
Search	Hero/product search navigation and client-side product card filtering.
Product UI	Card hover interactions and safe click behavior.
Forms	Focus/blur styling and submission feedback.
Images	Load/error handling and visual state management.
Notifications	Dynamic flash container and dismissible notification helper.
Scroll	Header/scroll-dependent interface behavior.
Currency	Indian currency formatting helper.

12.3 Template Inheritance

base.html master layout hai. Page templates Jinja2 extends/block pattern se title aur main content supply karte hain. `url_for()` static files aur application routes ke URLs generate karta hai, जिससे deployment path changes ke against links maintainable rehte hain.

13. Installation and Local Setup

13.1 Prerequisites

- Python 3.8 or newer (project guidance); Python 3.10+ recommended.
- MySQL Server and optional MySQL Workbench.
- Git for source control.
- Modern browser such as Chrome, Edge or Firefox.

13.2 Setup Procedure (Windows PowerShell)

1. Repository folder open karein.
2. Virtual environment create aur activate karein.
3. requirements.txt dependencies install karein.
4. database/schema.sql MySQL mein execute karein.
5. Ignored config/local.py mein database settings aur strong secret configure karein.
6. python app.py se server start karein.
7. Browser mein `http://localhost:5000` open karein.

```
cd C:\Users\ujaga\OneDrive\Desktop\plant-store
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
python app.py
```

13.3 Database Initialization

```
mysql -u root -p < database/schema.sql
```

MySQL Workbench alternative: File > Open SQL Script se database/schema.sql open karke complete script execute karein. Script database aur core tables create karta hai. Main app later catalog version synchronization aur payments/schema additions ensure karti hai.

13.4 Dependencies

Package	Version/range	Purpose
Flask	2.3.0	Web framework, routing, templates, session and request handling
PyMySQL	1.1.2	MySQL database client
cryptography	>=43.0.0	Secure cryptographic support used by dependency stack
Werkzeug	2.3.0	Password hashing and Flask utilities

14. Configuration and Security

14.1 Configuration Strategy

app.py development defaults define karta hai aur config/local.py ko silent load karta hai. local.py .gitignore se excluded hai, isliye machine-specific database password aur secret Git repository mein commit nahi hone chahiye.

```
MYSQL_HOST = 'localhost'
MYSQL_USER = '<database-user>'
MYSQL_PASSWORD = '<strong-password>'
MYSQL_DB = 'plant_store'
SECRET_KEY = '<long-random-secret>'
```

Credential safety: Documentation real local credentials intentionally include nahi karti. Agar koi secret kabhi public repository/log mein expose hua ho to usse rotate karna chahiye.

14.2 Production Hardening Checklist

- SECRET_KEY environment/secret manager se load karein; development fallback remove karein.
- Database user ko least-privilege grants dein; root account use na karein.
- CSRF protection, secure cookies (Secure, HttpOnly, SameSite) aur HTTPS enforce karein.
- Registration/login rate limiting aur stronger password policy add karein.
- Server-side validation phone/address lengths aur allowed payment method whitelist kare.
- Stock update ko atomic conditional UPDATE/transaction locking se race-safe banayein.
- Real payment gateway use karte waqt server-side signature/webhook verification implement karein.
- Public tracking endpoint par additional identity proof aur throttling add karein.
- Production WSGI server (Waitress/Gunicorn equivalent) aur reverse proxy use karein.
- Centralized logging, backups, monitoring and error reporting configure karein.

14.3 Data Handling

Application personal data mein name, email, phone aur delivery address store karti hai. Production operator ko access control, retention/deletion policy, encrypted backups, privacy notice accuracy aur applicable Indian data-protection obligations review karne chahiye.

15. Testing and Troubleshooting

15.1 Recommended Functional Test

1. Home page load karein aur nine category links verify karein.
2. Each category filter mein expected product cards aur images check karein.
3. New unique email register karke logout/login test karein.
4. Same product twice add karke quantity increment verify karein.
5. Quantity update/remove aur cart total verify karein.
6. Rs.999 threshold ke below/above shipping calculation test karein.
7. Invalid PIN and missing checkout fields reject hone chahiye.
8. Demo failed payment par order/stock/cart unchanged verify karein.
9. Demo successful UPI/card and COD flows separately test karein.
10. Order history, success page, tracking number and contact form verify karein.
11. Invalid product/policy/order success paths par 404 behavior check karein.
12. Mobile viewport mein navigation, filters, forms and cart layout inspect karein.

15.2 Troubleshooting Matrix

Problem	Likely cause	Resolution
Cannot connect to MySQL	Service stopped, wrong credentials/database	Start MySQL; verify local.py and plant_store schema.
Module not found	Environment inactive/dependencies missing	Activate venv and run pip install -r requirements.txt.
Image missing	Filename/path mismatch	Check static/uploads and catalog/database image value.
Port 5000 busy	Another server process	Stop process or configure a different app.run port.
Duplicate products	schema sample data rerun	Inspect products rows; use catalog sync as canonical source.
Checkout redirects	Not logged in, empty cart or missing session checkout	Login, add item and restart checkout.
Order not found	Incorrect number/case or missing committed order	Copy confirmation number; verify orders table.
500 error	Database/schema/runtime exception	Review terminal logs and confirm schema permissions.

15.3 Existing Automated Check

test_categories.py category/catalog behavior check karne ke liye repository mein present hai. Broader automated coverage ke liye pytest fixtures, test database, Flask test client aur transaction rollback-based isolated tests add karne chahiye.

16. Known Limitations and Roadmap

Priority	Current limitation	Recommended enhancement
High	Demo-only payment	Razorpay/Stripe/PayU sandbox integration with verified webhooks.
High	No CSRF protection	Flask-WTF and CSRF tokens across all state-changing forms.
High	Stock race possible	Atomic inventory update and transaction locking.
High	No admin console	Role-based product, inventory, order and contact management.
Medium	Simple client-side search	Server search, sort, price filter and pagination.
Medium	No password reset/email verification	Token-based email lifecycle.
Medium	Public tracking uses order number only	Email/phone confirmation and rate limiting.
Medium	Runtime schema changes	Alembic/Flask-Migrate versioned migrations.
Low	No wishlist/reviews	Customer engagement features with moderation.
Low	Limited SEO/analytics	Metadata, sitemap, structured data and privacy-aware analytics.

Recommended Delivery Phases

1. Stabilize: tests, CSRF, secrets, atomic stock, validation and migrations.
2. Operate: admin dashboard, order states, email notifications, logging and backups.
3. Transact: real gateway sandbox, webhook reconciliation, refunds and invoice generation.
4. Grow: search/filter/pagination, wishlist, reviews, SEO and performance optimization.

Project Strengths

- Complete customer shopping journey already demonstrated end-to-end.
- Clear separation of templates, static assets, application logic and curated catalog.
- Parameterized SQL, password hashing and per-user cart/order ownership checks.
- Broad category catalog with local image coverage and responsive storefront.
- Catalog/version and schema compatibility logic supports iterative development.

Appendix A. Folder Structure

plant-store/	
app.py	Main Flask application
catalog.py	Categories and curated catalog
requirements.txt	Python dependencies
config/local.py	Local secrets (ignored by Git)
database/schema.sql	MySQL schema and seed data
templates/	Jinja2 page templates
static/css/style.css	Main responsive stylesheet
static/js/main.js	Browser interactions/search
static/uploads/	Product and marketing images
images/	Source/reference image collection
test_categories.py	Catalog/category check
README.md	Project overview
SETUP_GUIDE.md	Local setup guidance

Template Inventory

Template group	Files
Shared	base.html, error.html
Storefront	index.html, products.html, product_detail.html
Account	login.html, register.html
Commerce	cart.html, checkout.html, payment.html, order_success.html, orders.html, track_order.html
Content/support	about.html, contact.html, faq.html, policy.html

Appendix B. Operational Checklist

- ☐ MySQL service running and plant_store accessible.
- ☐ Local configuration exists and is excluded from Git.
- ☐ Strong secret and least-privilege database user configured.
- ☐ Dependencies installed in active virtual environment.
- ☐ Catalog synchronization completes without errors.
- ☐ Product images render without broken links.
- ☐ Registration, login, cart and checkout smoke test passes.
- ☐ Payment disclaimer remains visible until real gateway is integrated.
- ☐ Database backup and restore procedure tested before production changes.
- ☐ Debug mode disabled and HTTPS/reverse proxy configured for deployment.

Document End

This documentation reflects the repository implementation reviewed on 03 October 2026. Future code changes should update route, database, configuration and operational sections together so the document remains a reliable source of truth.